Neurocomputing 367 (2019) 39–45

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Integration of residual network and convolutional neural network along with various activation functions and global pooling for time series classification

Xiaowu Zou[a], Zidong Wang[b], Qi Li[a], Weiguo Sheng[a,*]

[a] Department of Computer Science, Hangzhou Normal University, Hangzhou, PR China
[b] Department of Computer Science, Brunel University London, Uxbridge, Middlesex UB8 3PH, UK

ARTICLE INFO

Article history:
Received 27 May 2019
Revised 21 July 2019
Accepted 8 August 2019
Available online 9 August 2019

Communicated by Steven Hoi

Keywords:
Time series classification
Residual network
Convolutional neural network
Deep learning

ABSTRACT

In this paper, we devise a hybrid scheme, which integrates residual network with convolutional neural network, for time series classification. In the devised method, the architecture of network is constructed by facilitating a residual learning block at the first three convolutional layers to combine the strength of both methods. Further, different activation functions are used in different layers to achieve a decent abstraction. Additionally, to alleviate overfitting, the pooling operation is removed and the features are fed into a global average pooling instead of a fully connected layer. The resulting scheme requires no heavy preprocessing of raw data or feature crafting, thus could be easily deployed. To evaluate our method, we test it on 44 benchmark datasets and compare its performance with related methods. The results show that our method can deliver competitive performance among state-of-the-art methods.

© 2019 Elsevier B.V. All rights reserved.

1. Introduction

Time Series Classification (TSC) [1] involves training a classifier on a set of labeled data objects, which consist of an ordered set of real valued attributes, to predict class labels of unseen time series data. TSC is deemed to be a difficult task. This is due to time series does not have a clear feature, which make it difficult to generate an interpretable classifier. Further, time series data often contains a lot of noise and has a high dimensionality. To deal with this task, many methods have been proposed in literature and they generally can be divided into three categories [1,2]: distance-based, feature-based and ensemble-based methods. Distance-based methods [3] involve measuring the distance or similarity between given two time series data objects then using k-nearest neighbor method or support vector machines with kernels to perform classification. Dynamic Time Warping (DTW) [4] is a representative and perhaps most popular method of this category, in which dynamic warping is used to align time series data. DTW with warping set through cross-validation (DTWCV) [37] is also widely used as a benchmark method. The DTW combined with the k-nearest neighbor classifier has demonstrated to be efficient for TSC. However, this algorithm is computational expensive and is not able to handle applications, which require real-time feedback. Further, although the nearest neighbor classifier used in this method has the advantages of high classification accuracy and easy to be implemented, the results of which can only indicate the similarity of the objects in the same category, rather than the distance of different categories. Consequently, the classification results are difficult to be interpreted. In addition, the nearest neighbor classifier is a lazy classifier, which creates a corresponding classifier for each test instance, thus requiring a large amount of computational resource and space.

Feature-based methods [5], on the other hand, perform the classification by symbolizing a subsequence and extracting a set of features, which are used to represent global or local time series patterns. For example, Lin et al. [6] proposed such a method, called Symbolic Aggregate Approximation (SAX). This method reduces the dimension of time series data through piecewise aggregate approximation, then discretizes the resulting features into bins for classification. Senin and Malinchik [7] developed a method, called Symbolic Aggregate Approximation Vector Space Model (SAXVSM), which combines SAX representation with a vector space model for TSC. Schäfer [8] proposed a method, named Bag Of SFA Symbols (BOSS), which uses windows to form words over time series. This method differs with SAX and SAXVSM in that it uses a truncated Discrete Fourier Transform (DFT) to reduce the dimension and maintain most information of the time series. However, in this

* Corresponding author.
E-mail address: w.sheng@ieee.org (W. Sheng).

https://doi.org/10.1016/j.neucom.2019.08.023
0925-2312/© 2019 Elsevier B.V. All rights reserved.

method, many local features of the original data could be lost since the DFT is a global method. Further, in [9], Schäfer combined the BOSS with vector space (VS) to devise a method called BOSSVS. Deng et al. [10] designed a method named Time Series Forest (TSF). This method uses statistics of each interval as features. Baydogan et al. [11] extended the TSF and proposed a bag-of-features framework. The framework provides a comprehensive representation that handles both global and local features. The resulting framework can be used with any supervised learner for classification via the random forest classifier. Additionally, Grabocka et al. proposed a method called learning time series shapelets (LTS) [40]. This method is able to learn shapelets directly and achieves competitive results on classifying logistic regressions. Feature-based methods have been widely applied for TSC. However, this category of method generally requires a large number of handcrafted features and is difficult to implement due to the high dimensionality and complexity of time series data.

The third category, ensemble-based methods [12], performs TSC by assembling different classifiers. For instance, Lines and Bagnall [13] proposed the Elastic Ensemble (EE), which combines 11 nearest neighbor classifiers. These classifiers use whole series elastic distance measures in the time domain and with first order derivatives, then perform prediction via 1-NN classifiers based on a voting scheme. Bagnall et al. [14] developed the Shapelet Ensemble (SE), which produces the classifier through shapelet transformation with a varied ensemble. In this method, discriminative shapelet needs to be identified from a large space, which is very time-consuming. While, Bagnall et al. [14] proposed a method, called flat Collective of Transform-based Ensembles (flat-COTE). The method involves pooling 35 classifiers into a single ensemble with votes weighted by training set cross validation accuracy. The ensemble-based methods can give highly competitive performances on TSC. However, they suffer very expensive complexity during the training process as well as testing and are difficult to be applied for real applications.

Recently, Convolutional Neural Network (CNN) [15] based deep learning methods have also been proposed for TSC. These methods are promising as they can preserve the spatial structure of TSC problem and extract suitable internal structure to generate features for classification. However, as the depth of the network increases, the training could become very difficult. The problem can be alleviated by employing strategies, such as normalized initialization [16] and batch normalization [17]. While, when the deep network converges, it comes with another issue that as the depth of network continues to increase, the error of training data could increase rather than decrease. This issue is known as degradation problem and the Residual Network (ResNet) with shortcut connection could be helpful in this case [18]. Additionally, overfitting is occurred commonly in training process. Many schemes have been proposed to deal with this issue. Some recent prominent studies try to alleviate the problem by integrating regularization or bias into the training process [41–45]. What's more, the activation function in CNNs is crucial for their performances and existing methods are typically based on a single activation function. Since different activation functions possess different properties [26,28–30], it may be desirable to employ different activation functions in different layers of the CNN.

In this work, a deep learning method by combining the residual network with CNN is proposed to deal with TSC. In the proposed method, we employ residual learning blocks with a short cut connection to facilitate the degradation problem of CNN based deep networks. Further, to alleviate overfitting, the pooling operation is removed, and the features are fed into a global average pooling instead of a fully connected layer. Additionally, different activation functions are used in different layers to achieve a decent abstraction. The resulting method has been tested on 44 benchmark datasets and compared with related method. The experimental results show the proposed method can deliver better or comparable performance than related methods.

The remainder of the paper is organized as follows. Section 2 reviews related works. Then, Section 3 describes the proposed method. This is followed by experiments in Section 4. Finally, Section 5 summarizes the work.

2. Related work

In this section, we provide a brief review of CNN based methods for TSC as well as activation functions used in the CNN. In [19], a multi-channel CNN (MC-CNN) was proposed for multivariate time series classification. This method first extracts features by feeding time series data to different CNNs. Then, the extracted features are concatenated and fed into a new CNN framework. To train the CNN architecture, large multivariate datasets are required for this method. In [20], a multi-scale CNN (MCNN) was developed to cope with univariate time series. In this method, two additional components, which are used to extract multi-scale and multi-frequency information, are introduced to improve the classification accuracy. The method has shown to be effective, however it requires heavy preprocessing efforts and tedious hyperparameters tuning. In [21], Wang et al. provided a baseline for time series classification by combining multilayer perceptron, fully connected network and residual network. The results show that the global average pooling process can enhance the exploitation capability of convolutional model for the map of class activation, thus appropriately identifying the contributing region in the data [21].

The above CNN models have been served as a good starting point for addressing TSC and have been further improved from various aspects. For example, Hinton et al. [15] proposed several data augmentation techniques to alleviate the relative scarcity of the time series data. Sutskever et al. [22] designed a network initialization method, which can avoid the issue of vanishing gradient while achieving an efficient convergence for training. Ioffe and Szegedy [17] proposed a batch normalization schema to speed up the convergence and help improve the generalization performance. He et al. [18] introduced a residual learning block to ease the training of substantially deeper networks. The authors explicitly reformulate the layers as learning residual functions with reference to their inputs. In this method, the sum of activation of a residual function as well as a shallower unit is used for activation of deeper units. By doing so, the gradients can directly propagate to the units of shallower level. As a result, the deep network can be easily optimized. He et al. [23] proposed another pre-activation variant of residual network. Their results show that the proposed networks can easily learn the identity shortcut connection. The work also shows that bringing batch normalization forward can have a significant better performance than employing batch normalization after addition. The experiments show that the residual network with batch normalization and Rectified Linear Unit (ReLU) pre-activation obtains higher accuracies than exiting residual networks [23]. Shen et al. [24] proposed to gradually introduce the trainable layers by using weighting factors from convolutional layers for the outputs. Further, a recent work of Inception-v4 [25] reported that employing identity skip connections across inception modules could accelerate the training as well as improve the performance. Indeed, shortcut connections have been widely employed to train deep networks [16,23,25]. However, for deep networks, especially those with various residual network blocks, the training could be complex and time consuming.

Activation function, which introduces nonlinearities, is a key module in deep CNNs. Various activation functions have been proposed in literature. For example, Nair and Hinton [26] proposed a ReLU activation function. This function allows networks to

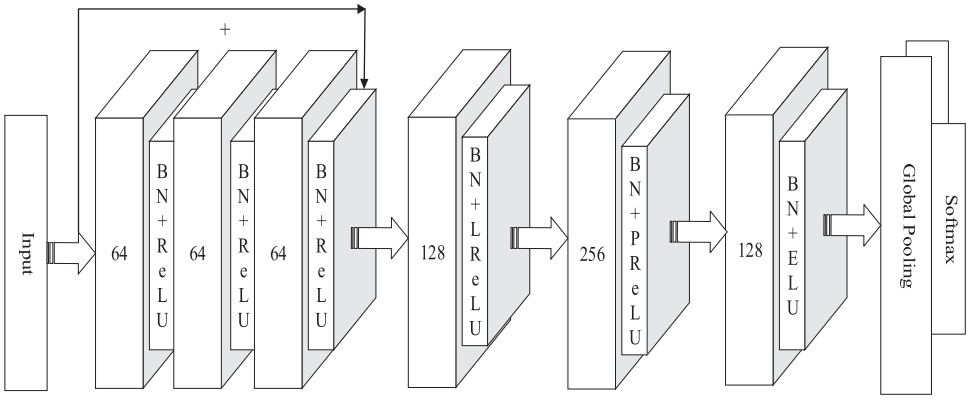


Fig. 1. The overall architecture of proposed method.

acquire sparse representations by inducing the sparsity in hidden units and can be used to be efficiently train deep networks [27]. However, ReLU unit could have zero gradient no matter when it is active or not, which is termed "dying ReLU" problem [26]. To address this issue, Mass et al. [28] introduced an activation function called Leaky ReLU (LReLU). Rather than using a predefined parameter in LReLU activation function, He et al. [29] proposed a Parametric ReLU (PReLU) activation function, which adaptively learns the parameters of rectifiers. As PReLU activation function involves very few extra parameters, the overfitting risk is low and requiring very little additional computational cost. While, in [30], Exponential Linear Unit (ELU) activation function was introduced. This function can be used to enable a fast learning of deep CNNs through a negative part. Undoubtedly, an appropriate activation function can greatly improve the CNN's performance on certain tasks. However, to achieve a proper performance, it may be desirable to deploy different activation functions in different layers.

3. Proposed method

In this section, we present the proposed method, which combines the residual network with CNN (Res-CNN) for TSC. Further, in the proposed method, the pooling operation is removed and the fully connected layer is replaced by global average pooling to reduce the number of parameters as well as to improve the performance. Additionally, we leverage batch normalization technique and exploit diverse activation functions in different layers to enhance the nonlinear abstraction capacity. The resulting scheme requires no heavy preprocessing of raw data or feature crafting, thus could be easily deployed. The overall architecture of proposed method is shown in Fig. 1.

Specifically, in the proposed method, the Res-CNN is constructed by facilitating a residual learning block with shortcut connection technique at the first three convolutional layers. ResNet with shortcut connection technique can be used to learn highly complex patterns in the data. However, it generally involves tremendous computation for training and could easily lead to overfitting issue. While, the CNN could be efficient and can be used to effectively learn the temporal and spatial patterns from raw data, but it typically lacks of the capability to recover the complex patterns due to it involves few levels of network. Here, we therefore tend to combine the strength of ResNet and CNN to deal with TSC.

Further, a neural network without an activation function is essentially a linear regression algorithm. The activation function decides whether a neuron should be activated or not by calculating weighted sum and adding bias, leading to a non-linear learning. This, in turn, makes it capable of performing complex learning tasks. For this purpose, various activation functions have been proposed. Among these functions, the ReLU is perhaps the most widely used due to its fast learning performance. On the other hand, LReLU, PReLU and ELU are the improved version of the ReLU in different perspectives, which could enhance each other. Therefore, we propose to employ activation functions including ReLU, LReLU, PReLU and ELU at different layers to achieve a decent abstraction. Comparing to other blocks, the residual learning block generally requires intensive computational resources for training. To improve its efficiency, ReLU activation function is first employed in the residual learning block. The ReLU does not require expensive operations, such as exponentials, by simply thresholding a matrix of activations at zero and can accelerate the convergence of model due to its non-saturating nonlinearity. However, it suffers from the issue of zero gradients when the unit is not active, i.e., the weights of units will not be adjusted if they are not initially activated. To alleviate this issue, LReLU activation function is then used at the layer after the residual learning block. This function has a slight slope in the negative range and can be used to alleviate the issue mentioned above. In the subsequent layer, PReLU activation function is used to adaptively learn the parameters of rectifiers. While, in last layer, we employ the ELU activation function. Unlike other functions, the ELU can produce negative outputs, which is beneficial for fast learning and robust to noise. Such a function does not lead to a small derivative and can also be used to avoid the vanishing gradient problem, which arises when a large input space is mapped to a small one.

Additionally, we removed the pooling operation, which is typically used in the residual network. The reason is that although pooling operation can bring invariances to local translation, it ignores the precise locations of the features, which is critical for TSC problems. Further, the pooling operation will reduce the number of inputs to the next layer of feature extraction, thus limiting the information, which could be passed to the next layer. Also, in the proposed method, we leverage batch normalization before activation. By doing so, the proposed method can be less sensitive to initialization while high learning

Finally, we adopt a global average pooling layer rather than the fully connected layer. This is due to the global average pooling layer can generate one feature map for each category and the average of each feature map can be calculated. These average values serve as the probability of each category. By contrast, the fully connected layer can be used to reduce the dimension of data. However, it is prone to overfitting and depends heavily on dropout regularization. More importantly, fully connected layer fails to consider the correspondence between feature maps and categories [31].

The time complexity and spatial complexity of the proposed method are as follows. The time complexity of the convolutional

neural network is $O(\sum_{l=1}^{D} M_l^2 K_l^2 C_{l-1} C_l)$, and the space complexity is $O(\sum_{l=1}^{D} K_l^2 C_{l-1} C_l + \sum_{l=1}^{D} M^2 C_l)$. Here, D is the number of convolutional layers of the neural network, that is, the depth of the network. M is the side length of the feature map for the convolution kernel. K is the side length of each convolution kernel. l is the lth convolutional layer of the neural network, and C_l is the number of output channels of the lth convolutional layer of the neural network, that is, the number of convolution kernels of the layer. For the lth convolutional layer, the number of input channels is the number of output channels of the $l1$th convolutional layer. The proposed model fuses the residual network and the convolutional neural network, which contains 6 convolutional layers. It removes the pooling operation and replaces the fully connected layer with the global average pooling, which reduces the number of operations and the number of parameters.

4. Experiments

We first perform two sets of experiments to evaluate the effectiveness of global pooling operation and activation functions in the proposed method. Then, experiments are carried out compare the performance of proposed method with related methods. All experiments are implemented on a Lenovo Laptop with Intel Core i7 CPU (2.40 GHz), 8 G memory and the open machine learning framework, TensorFlow, is used as the platform.

The data sets used in the experiments are from the UCR time series repository [32], which includes 44 distinct time series datasets. These data sets are from six domains including *image, motion, sensor, ECG, stimulated* and *Spectro*. In these data sets, the length of time series ranges from 24 to 1882, the size of training set varies from 16 to 1800, the size of testing set is from 20 to 6164 while the number of classes is between 2 and 42. The data sets are normalized using the z-normalization technique, which has been used in [33].

The setting of proposed method is as follows. The convolution operation is fulfilled by three 1-D kernels using the sizes of {8, 5, 3} and without striding. The number of filters is {64, 128, 256, 128}. The parameters of the activation functions LReLU and ELU are set to be 0.2 and 0.3, respectively. The networks are trained using Adam [34] with the learning rate of 1.0e−4. The loss function is based on the categorical cross entropy [35]. The optimal model, in which the training loss is first minimized and then evaluated on the testing data sets, is used in the experiments. To train the network, it has two steps. The first step is to train the layers from bottom to top, which can be regarded as feature learning process. In this step, the first layer is trained with unlabeled raw data, whose outputs are used to train the next layer and so on. The parameters of each layer are obtained separately by training all the layers in this step. Based on the parameters obtained in the first step, the second step is then to fine-tune them by training the network with labeled data.

To show the effect of pooling operation and activation functions used in the proposed method, we compare the performance of proposed method (denoted as Res-CNN) against its two variants: (1) the proposed method using the traditional pooling operation (Denoted as Res-CNN_1) and (2) instead of using different activation functions at different layers, the same activation function is used for all layers (Denoted as Res-CNN_2).

All of the three methods are implemented using the same setting and tested on 12 out of 44 UCR data sets (two representative data sets from each domain). The results of the above two pairwise comparisons are shown in Tables 1 and 2, respectively. The best result is highlighted in bold. From Table 1, we can see that by removing the pooling operation and replacing the fully connected with the global average pooling is able to improve the classification accuracy. Similar results can also be observed from Table 2 by comparing the Res-CNN and Res-CNN_2. It therefore can be concluded that by using various activation functions on different layers can significantly improve the performance of the proposed method.

Table 1
Comparing the testing errors of the proposed method and its variant (i.e., using the traditional pooling operation) on 12 data sets.

Data sets	Domain	Res-CNN_1	Res-CNN
Adiac	image	0.16	**0.148**
FaceFour	image	0.08	**0.034**
TwoPattern	stimulated	0.006	**0**
Synthetic_Control	stimulated	0	**0**
OliveOil	spectro	0.14	**0.066**
Coffee	spectro	0	**0**
TwoLeadECG	ECG	0	**0**
ECGFiveDay	ECG	**0.002**	0.009
uWaveGestureLibrary_X	motion	0.223	**0.216**
GunPoint	motion	**0**	0.006
ItalyPowerDemand	sensor	0.04	**0.034**
Wafer	sensor	0.002	**0.001**

Table 2
Comparing the testing errors of the proposed method and its variants (i.e., using the same activation function for all layers) on 12 data sets.

Data sets	Res-CNN_2				Res-CNN
	ReLU	LReLU	PReLU	ELU	
Adiac	0.162	0.154	0.154	0.162	**0.148**
FaceFour	0.046	0.046	0.046	0.08	**0.034**
TwoPattern	0.029	0.019	0.028	0.02	**0**
Synthetic_Control	**0**	0.004	**0**	0.004	**0**
OliveOil	0.2	0.267	0.167	0.267	**0.066**
Coffee	**0**	**0**	**0**	**0**	**0**
TwoLeadECG	**0**	**0**	**0**	**0**	**0**
ECGFiveDay	0.01	0.005	0.019	**0.002**	0.009
uWaveGestureLibrary_X	0.22	0.217	0.22	0.22	**0.216**
GunPoint	0.014	**0**	0.007	**0**	0.006
ItalyPowerDemand	0.035	0.035	0.039	0.043	**0.034**
Wafer	0.003	0.002	0.002	0.002	**0.001**

Then, we evaluate our method against related methods. The related methods to be compared including 1-NN with Euclidean Distance (ED) [36], DTW [4], DTWCV [37], SAX with Vector space model (SV) [7], Fast Shapelet (FS) [38], Shotgun Classifier (SC) [39], Bag-of-SFA-Symbols (BOSS) [8], Time Series based on a Bag-of-Features (TSBF) [11], Time Series Forest (TSF) [10], 1-NN Bag-Of-SFA-Symbols in Vector Space (BOSSVS) [9], Elastic Ensemble (PROP) [13], Shapelet Ensemble (SE) model [14], Learn Shapelets Model (LTS) [40], flat-COTE (COTE) [14], multi-scale CNN (MCNN) [20], ResNet [21] and fully convolutional network (FCN) [21]. All of the 44 UCR data sets have been used for testing. Their performances are evaluated using the metrics of the classification accuracy and mean prediction.

Table 3 shows the results of misclassification rates of the 18 methods to be compared on the 44 data sets. On each dataset, we also rank all methods and report the average ranking results. The mean ranks as well as mean errors are given in the last two rows of the table and have also been visually shown in Fig. 2. From the results, it can be found that our proposed method achieves the best accuracy on 19 data sets out of 44. Especially, our proposed method gives a mean error of 0.129, which is better than all of the methods to be compared, except for the ensemble method COTE. In terms of mean rank, our method is 4.6, which places our method in the third place after the COTE and MCNN. Although MCNN could achieve better mean rank than our method, it requires heavy preprocessing efforts and a large of set of hyperparameters tuning. From Fig. 2, the results clearly show that our proposed method along with COTE and MCNN are top three methods with

Table 3
Comparing the performance of 18 methods on 44 UCR data sets.

Data sets	DTW	ED	DTWCV	FS	SV	BOSS	SE	TSBF	TSF	BOSSVS	PROP	LTS	SC	COTE	MCNN	ResNet	FCN	Res-CNN
Adiac	0.396	0.389	0.389	0.514	0.417	0.22	0.373	0.245	0.261	0.302	0.353	0.437	0.435	0.233	0.231	0.179	0.21	**0.148**
Beef	0.367	0.467	0.333	0.447	0.467	0.2	**0.133**	0.287	0.3	0.267	0.367	0.24	0.167	0.133	0.367	0.233	0.367	0.3
CBF	0.003	0.148	0.006	0.053	0.007	**0**	0.01	0.009	0.039	0.001	0.002	0.006	0.003	0.001	0.002	0.006	0.008	0.005
ChrlorineCon	0.352	0.35	0.35	0.417	0.334	0.34	0.312	0.336	0.26	0.345	0.36	0.349	0.3	0.314	0.203	0.172	0.157	**0.156**
CinCECGTorso	0.349	0.103	0.07	0.174	0.334	0.125	**0.021**	0.262	0.069	0.13	0.062	0.167	0.154	0.064	0.058	0.229	0.178	0.191
Coffee	**0**	**0**	**0**	0.068	**0**	**0**	**0**	0.004	0.071	0.036	**0**	**0**	**0**	**0**	0.036	**0**	**0**	**0**
CricketX	0.246	0.423	0.228	0.527	0.308	0.259	0.297	0.278	0.287	0.346	0.203	0.209	0.218	**0.154**	0.182	0.226	0.208	0.197
CricketY	0.256	0.433	0.238	0.505	0.318	0.208	0.326	0.259	0.2	0.328	0.156	0.249	0.236	0.167	**0.154**	0.195	0.221	0.223
CricketZ	0.246	0.413	0.254	0.547	0.297	0.246	0.277	0.263	0.239	0.313	0.156	0.2	0.228	**0.128**	0.142	0.205	0.187	0.21
DiatomSizeR	0.033	0.065	0.065	0.117	0.121	0.046	0.069	0.126	0.101	0.036	0.059	0.033	0.124	0.082	**0.023**	0.069	0.07	0.068
ECGFiveDays	0.232	0.203	0.203	0.004	0.003	**0**	0.055	0.183	0.07	**0**	0.178	0	0.001	**0**	**0**	0.013	0.015	0.009
FaceAll	0.192	0.286	0.192	0.411	0.244	0.21	0.247	0.234	0.231	0.241	0.152	0.217	0.263	0.105	0.235	0.167	**0.046**	0.154
FaceFour	0.17	0.216	0.114	0.09	0.114	**0**	0.034	0.051	0.034	0.034	0.091	0.048	0.057	0.091	**0**	0.045	0.068	0.034
FacesUCR	0.095	0.231	0.088	0.328	0.1	**0.042**	0.079	0.09	0.109	0.103	0.063	0.059	0.087	0.057	0.063	0.046	0.052	0.049
fiftywords	0.31	0.369	0.235	0.489	0.374	0.301	0.288	0.209	0.277	0.367	**0.18**	0.232	0.281	0.191	0.19	0.288	0.321	0.303
fish	0.177	0.217	0.154	0.197	0.017	**0.011**	0.057	0.08	0.154	0.017	0.034	0.066	0.023	0.029	0.051	0.017	0.029	0.022
GunPoint	0.093	0.087	0.087	0.061	0.013	**0**	0.06	0.011	0.047	**0**	0.007	**0**	0.02	0.007	**0**	0.007	**0**	0.006
Haptics	0.623	0.63	0.588	0.616	0.575	0.536	0.607	0.488	0.565	0.584	0.584	0.532	0.523	0.488	0.53	0.487	0.516	**0.477**
InlineSkate	0.616	0.658	0.613	0.734	0.593	**0.511**	0.653	0.603	0.675	0.573	0.567	0.573	0.615	0.551	0.618	0.635	0.687	0.627
ItalyPower	0.05	0.045	0.045	0.095	0.089	0.053	0.053	0.096	0.033	0.086	0.039	**0.03**	0.048	0.036	**0.03**	0.04	0.036	0.034
Lightning2	0.131	0.246	0.131	0.295	0.23	0.148	**0.098**	0.257	0.18	0.262	0.115	0.177	0.344	0.164	0.164	0.246	0.279	0.262
Lightning7	0.274	0.425	0.288	0.403	0.342	0.342	0.274	0.262	0.263	0.288	0.233	0.197	0.26	0.247	0.219	0.151	**0.137**	**0.137**
MALLAT	0.066	0.086	0.086	0.033	0.199	0.058	0.092	0.037	0.072	0.064	0.05	0.046	0.06	0.036	0.057	0.021	**0.02**	**0.02**
MedicalImages	0.263	0.316	0.253	0.433	0.516	0.288	0.305	0.269	0.232	0.474	0.245	0.27	0.396	0.258	0.26	0.224	0.216	**0.209**
MoteStrain	0.165	0.121	0.134	0.217	0.117	0.073	0.113	0.135	0.118	0.115	0.114	0.087	0.109	0.085	0.079	0.076	0.073	**0.072**
NonInvThorax1	0.21	0.171	0.189	0.171	/	0.161	0.174	0.138	0.103	0.169	0.178	0.131	0.1	0.093	0.064	0.052	0.039	**0.036**
NonInvThorax2	0.135	0.12	0.12	0.12	/	0.101	0.118	0.13	0.094	0.118	0.112	0.089	0.097	0.073	0.06	0.049	0.045	**0.042**
OliveOil	0.167	0.133	0.133	0.213	0.133	0.1	0.133	0.09	0.1	0.133	0.133	0.56	0.1	0.1	0.133	0.267	0.167	**0.066**
OSULeaf	0.409	0.483	0.388	0.359	0.153	**0.012**	0.273	0.329	0.426	0.074	0.194	0.182	0.285	0.145	0.271	0.021	**0.012**	**0.012**
SonyAIBORobot	0.275	0.305	0.304	0.314	0.306	0.321	0.238	0.175	0.235	0.265	0.293	0.103	0.067	0.146	0.23	**0.015**	0.032	0.03
SonyAIBORobotII	0.169	0.141	0.141	0.215	0.126	0.098	0.066	0.196	0.177	0.188	0.124	0.082	0.115	0.076	0.07	0.038	0.038	**0.036**
StarLightCurves	0.093	0.151	0.095	0.06	0.108	**0.021**	0.093	0.022	0.036	0.096	0.079	0.033	0.024	0.031	0.023	0.029	0.033	0.031
SwedishLeaf	0.208	0.213	0.154	0.269	0.275	0.072	0.12	0.075	0.109	0.141	0.085	0.087	0.093	0.046	0.066	0.042	0.034	**0.03**
Symbols	0.05	0.1	0.062	0.068	0.089	0.032	0.083	0.034	0.121	**0.029**	0.049	0.036	0.114	0.046	0.049	0.128	0.032	0.119
SyntheticControl	0.007	0.12	0.017	0.081	0.013	0.03	0.033	0.008	0.023	0.04	0.01	0.007	0.17	**0**	0.003	**0**	0.01	**0**
Trace	**0**	0.24	0.01	0.002	**0**	**0**	0.05	0.02	**0**	**0**	0.01	**0**	0.02	0.01	**0**	**0**	**0**	**0**
TwoLeadECG	**0**	0.09	0.002	0.113	0.004	0.004	0.029	0.001	0.112	0.015	**0**	0.003	0.004	0.015	0.001	**0**	**0**	**0**
TwoPatterns	0.096	0.253	0.132	0.09	0.011	0.016	0.048	0.046	0.053	0.001	0.067	0.003	0.059	**0**	0.002	**0**	0.103	**0**
UwaveX	0.272	0.261	0.227	0.293	0.324	0.241	0.248	**0.164**	0.213	0.27	0.199	0.2	0.216	0.196	0.18	0.22	0.246	0.216
UwaveY	0.366	0.338	0.301	0.392	0.364	0.313	0.322	**0.249**	0.288	0.364	0.283	0.287	0.303	0.267	0.268	0.327	0.364	0.346
UwaveZ	0.342	0.35	0.322	0.364	0.357	0.312	0.346	**0.217**	0.267	0.336	0.29	0.268	0.273	0.265	0.232	0.256	0.275	0.248
wafer	0.02	0.005	0.005	0.004	0.002	**0.001**	0.002	0.004	0.047	0.001	0.003	0.004	0.002	0.001	0.002	0.003	0.003	**0.001**
WordSynonyms	0.351	0.382	0.252	0.563	0.436	0.345	0.357	0.302	0.381	0.439	**0.226**	0.34	0.403	0.266	0.276	0.392	0.402	0.4
youga	0.164	0.17	0.156	0.249	0.151	**0.081**	0.159	0.149	0.157	0.169	0.121	0.15	0.195	0.113	0.112	0.12	0.155	0.136
#best	3	1	1	0	1	13	4	3	1	5	4	5	1	5	8	6	8	**19**
mean error	0.205	0.249	0.185	0.266	0.214	0.143	0.176	0.169	0.178	0.185	0.155	0.159	0.169	**0.125**	0.135	0.135	0.139	0.129
mean rank	10.23	12.23	9.068	12.53	10.28	5.978	8.844	7.489	8.289	8.933	5.911	5.804	7.733	**4.0**	4.111	4.911	5.733	4.6

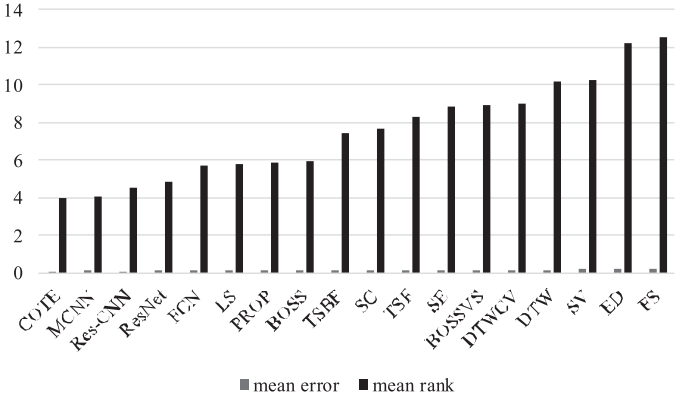


Fig. 2. The performance in terms of mean error and mean rank of 18 methods to be compared.

comparable performance in terms of mean error and mean rank, thus establishing the significance of our proposed method.

5. Conclusions

In this work, we propose and implement a synthesis of residual network with CNN for TSC. Further, in the proposed method, pooling operation is removed and a global average pooling is used instead of fully connection to alleviate overfitting. Additionally, different activation functions are employed in different layers to achieve a decent abstraction. The resulting method has been evaluated on UCR data sets and compared with related methods. The results show that the proposed method can deliver better or comparable performance than related methods. The results also confirm the significance of employing global pooling operations and different activation functions on different layers.

For future work, it would be desirable to integrate, for example, the Long Short-Term Memory (LSTM) network with CNN for TSC. The LSTM is able to achieve an appropriate abstraction of input observations over time, which could help CNN to improve the performance for TSC. Further, a light cascaded network, which is efficient and requires low memory, could also be incorporated to train parameters in all stages in our method thus improve the performance of the proposed method further.

Declaration of Competing Interest

None.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 61573316 and Grant 61873082.

References

[1] A. Bagnall, J. Lines, A. Bostrom, J. Large, E. Keogh, The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances, Data Min. Knowl. Discov. 31 (3) (2017) 606–660.
[2] Z. Xing, J. Pei, E. Keogh, A brief survey on sequence classification, ACM SIGKDD Explor. Newsl. 12 (1) (2010) 40–48.
[3] X. Wang, A. Mueen, H. Ding, G. Trajcevski, P. Scheuermann, E. Keogh, Experimental comparison of representation methods and distance measures for time series data, Data Min. Knowl. Discov. 26 (2) (2013) 275–309.
[4] H. Ding, G. Trajcevski, P. Scheuermann, X. Wang, E. Keogh, Querying and mining of time series data: experimental comparison of representations and distance measures, Proc. VLDB Endow. 1 (2) (2008) 1542–1552.
[5] A. Nanopoulos, R. Alcock, Y. Manolopoulos, Feature-based classification of time-series data, Int. J. Comput. Res. 10 (3) (2001) 49–61.
[6] J. Lin, E. Keogh, L. Wei, S. Lonardi, S.A.X. Experiencing, a novel symbolic representation of time series, Data Min. Knowl. Discov. 15 (2) (2007) 107–144.
[7] P. Senin, S. Malinchik, SAX-VSM: interpretable time series classification using sax and vector space model, in: Proceedings of the IEEE 13th International Conference on Data Mining, IEEE, 2013, pp. 1175–1180.
[8] P. Schäfer, The boss is concerned with time series classification in the presence of noise, Data Min. Knowl. Discov. 29 (6) (2015) 1505–1530.
[9] P. Schäfer, Scalable time series classification, Data Min. Knowl. Discov. 30 (5) (2016) 1273–1298.
[10] H. Deng, G. Runger, E. Tuv, M. Vladimir, A time series forest for classification and feature extraction, Inf. Sci. (Ny) 239 (2013) 142–153.
[11] M.G. Baydogan, G. Runger, E. Tuv, A bag-of-features framework to classify time series, IEEE Trans. Pattern Anal. Mach. Intell. 35 (11) (2013) 2796–2802.
[12] A. Bagnall, L. Davis, J. Hills, J. Lines, Transformation based ensembles for time series classification, in: Proceedings of the SIAM International Conference on Data Mining, 2012, pp. 307–318.
[13] J. Lines, A. Bagnall, Time series classification with ensembles of elastic distance measures, Data Min. Knowl. Discov. 29 (3) (2015) 565–592.
[14] A. Bagnall, J. Lines, J. Hills, A. Bostrom, Time-series classification with COTE: the collective of transformation-based ensembles, IEEE Trans. Knowl. Data Eng. 27 (9) (2016) 2522–2535.
[15] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, Adv. Neural Inf. Process. Syst. (2012) 1097–1105.
[16] R.K. Srivastava, K. Greff, J. Schmidhuber, Training very deep networks, in: Proceedings of the Advances Neural Information Processing Systems, 2015, pp. 2377–2385.
[17] S. Ioffe, C. Szegedy, Batch normalization: accelerating deep network training by reducing internal covariate shift, arXiv:1502.03167 (2015).
[18] K. He, X. Zhang, S. Ren, J. Sun, Identity mappings in deep residual networks, in: Proceedings of the European Conference on Computer Vision, 2016, pp. 630–645.
[19] Y. Zheng, Q. Liu, E. Chen, Y. Ge, J.L. Zhao, Time series classification using multi-channels deep convolutional neural networks, in: Proceedings of the International Conference on Web-Age Information Management, 2014, pp. 298–310.
[20] Z. Cui, W. Chen, Y. Chen, Multi-scale convolutional neural networks for time series classification, arXiv:1603.06995 (2016).
[21] Z. Wang, W. Yan, T. Oates, Time series classification from scratch with deep neural networks: a strong baseline, in: Proceedings of the International Joint Conference on Neural Networks (IJCNN), 2017, pp. 1578–1585.
[22] I. Sutskever, J. Martens, G.E. Dahl, G.E. Hinton, On the importance of initialization and momentum in deep learning, in: Proceedings of the International Conference on Machine Learning, 2013, pp. 1139–1147.
[23] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[24] F. Shen, R. Gan, G. Zeng, Weighted residuals for very deep networks, in: Proceedings of the 3rd International Conference on Systems and Informatics, 2016, pp. 936–941.
[25] C. Szegedy, S. Ioffe, V. Vanhoucke, A.A. Alemi, Inception-v4, inception-resnet and the impact of residual connections on learning, in: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, 2017.
[26] V. Nair, G. Hinton, Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th International Conference on Machine Learning (ICML-10), 2010, pp. 807–814.
[27] X. Glorot, A. Bordes, Y. Bengio, Deep sparse rectifier neural networks, in: Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, 2011, pp. 315–323.
[28] A.L. Maas, A.Y. Hannun, A.Y. Ng, Rectifier nonlinearities improve neural network acoustic models, in: Proceedings of the International Conference on Machine Learning, 30, 2013, pp. 3–9.
[29] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: surpassing human-level performance on imagenet classification, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 1026–1034.
[30] D.A. Clevert, T. Unterthiner, S. Hochreiter, Fast and accurate deep network learning by exponential linear units (ELUs), arXiv:1511.07289 (2015).
[31] M. Lin, Q. Chen, S. Yan, Network in network, arXiv:1312.4400 (2013).
[32] C. Yanping, E. Keogh, B. Hu, N. Begum, A. Bagnall, A. Mueen, G. Batista, UCR time series classification archive, URL www.Cs.Ucr.Edu/~eamonn/Time_series_data/. (2015).
[33] J. Paparrizos, L. Gravano, k-shape: efficient and accurate clustering of time series, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2015, pp. 1855–1870.
[34] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, arXiv:1412.6980 (2014).
[35] R.E. Hoskisson, M.A. Hitt, R.A. Johnson, D.D. Moesel, Construct validity of an objective (entropy) categorical measure of diversification strategy, Strateg. Manag. J. 14 (3) (1993) 215–235.
[36] G.E.A.P.A. Batista, X. Wang, E.J. Keogh, A complexity-invariant distance measure for time series, in: Proceedings of the SIAM International Conference on Data Mining, 2011, pp. 699–710.
[37] T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, E. Keogh, Data mining a trillion time series subsequences under dynamic time warping, in: Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence, 2013.
[38] T. Rakthanmanon, E. Keogh, Fast shapelets: a scalable algorithm for discovering time series shapelets, in: Proceedings of the SIAM International Conference on Data Mining, 2013, pp. 668–676.
[39] P. Schäfer, Towards time series classification without human preprocessing, in: Proceedings of the International Workshop on Machine Learning and Data Mining in Pattern Recognition, Springer, Cham, 2014, pp. 228–242.
[40] J. Grabocka, N. Schilling, M. Wistuba, L. Schmidt-Thieme, Learning time-series shapelets, in: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery And Data Mining, 2014, pp. 392–401.
[41] X. Luo, H. Wu, H. Yuan, M. Zhou, Temporal pattern-aware qos prediction via biased non-negative latent factorization of tensors, IEEE Trans. Cybern. (2019), doi:10.1109/TCYB.2019.2903736.
[42] X. Luo, M. Zhou, S. Li, Y. Xia, Z. You, Q. Zhu, H. Leung, Incorporation of efficient second-order solvers into latent factor models for accurate prediction of missing QoS data, IEEE Trans. Cybern. 48 (4) (2018) 1216–1228.
[43] X. Luo, M. Zhou, S. Li, M. Shang, An inherently nonnegative latent factor model for high-dimensional and sparse matrices from industrial applications, IEEE Trans. Ind. Inf. 14 (5) (2018) 2011–2022.
[44] X. Luo, M. Zhou, S. Li, Z. You, Y. Xia, Q. Zhu, A nonnegative latent factor model for large-scale sparse matrices in recommender systems via alternating direction method, IEEE Trans. Neural Netw. Learn. Syst. 27 (3) (2016) 579–592.
[45] X. Luo, J. Sun, Z. Wang, S. Li, M. Shang, Symmetric and nonnegative latent factor models for undirected, high-dimensional, and sparse networks in industrial applications, IEEE Trans. Ind. Inf. 13 (6) (2017) 3098–3107.



Xiaowu Zou received the B.Sc. Degree in computer science from Changchun University of Technology, Changchun, China, in 2016 and M.Sc. degree in Computer Science from Zhejiang University of Technology, Hangzhou, China, in 2019. He current works as a project assistant at Hangzhou Normal University.

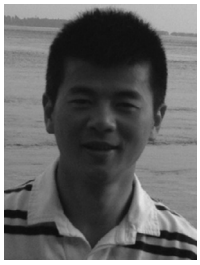


Zidong Wang (SM'03-F'14) was born in Jiangsu, China, in 1966. He received the B.Sc. degree in mathematics in 1986 from Suzhou University, Suzhou, China, and the M.Sc. degree in applied mathematics in 1990 and the Ph.D. degree in electrical engineering in 1994, both from Nanjing University of Science and Technology, Nanjing, China. He is currently Professor of Dynamical Systems and Computing in the Department of Information Systems and Computing, Brunel University London, U.K. From 1990 to 2002, he held teaching and research appointments in universities in China, Germany and the UK. Prof. Wang's research interests include dynamical systems, signal processing, bioinformatics, control theory and applications. He has published more than 300 papers in refereed international journals. He is a holder of the Alexander von Humboldt Research Fellowship of Germany, the JSPS Research Fellowship of Japan, William Mong Visiting Research Fellowship of Hong Kong. Prof. Wang serves (or has served) as the Editor-in-Chief for Neurocomputing and an Associate Editor for 12 international journals, including IEEE Transactions on Automatic Control, IEEE Transactions on Control Systems Technology, IEEE Transactions on Neural Networks, IEEE Transactions on Signal Processing, and IEEE Transactions on Systems, Man, and Cybernetics - Part C. He is a Fellow of the IEEE,

a Fellow of the Royal Statistical Society and a member of program committee for many international conferences.



Qi Li received her B.Eng. degree in electrical engineering and automation from Jiangsu University of Technology, Changzhou, China, in 2013 and the Ph.D. degree in control science and engineering from Donghua University, Shanghai, China, in 2018. She is currently a lecturer with the Institute of Service Engineering, Hangzhou Normal University, Hangzhou, China. From June 2016 to July 2016, she was a Research Assistant in the Department of Mathematics, Texas A&M University at Qatar, Qatar. From November 2016 to November 2017, she was a Visiting Ph.D. Student in the Department of Computer Science, Brunel University London, U.K. Her current research interests include network communication, complex networks and sensor networks. She is a very active reviewer for many international journals.



Weiguo Sheng received the M.Sc. degree in information technology from the University of Nottingham, U.K., in 2002 and the Ph.D. degree in computer science from Brunel University, U.K., in 2005. Then, he worked as a Researcher at the University of Kent, U.K. and Royal Holloway, University of London, U.K. He is currently a Professor at Hangzhou Normal University. His research interests include evolutionary computation, data mining/clustering, pattern recognition and machine learning.